Engineering Semantics for Trustworthy Industrial AI Agents: A State-of-the-Art Survey and Capability Framework

Anand Todkar, Jitendra Solanki, Mrinmoy Sarkar

***Abstract*—Industrial AI agents increasingly read plant data and propose actions, yet language-model reasoning alone does not establish which equipment is meant, in which unit a value is expressed, whether a command is legal in the current state, whether a guard is satisfied, or whether the evidence is current. Industrial information standards and semantic technologies supply much of this knowledge, but surveys have treated industrial language models, knowledge graphs, information models and semantic interoperability separately. This survey analyses the field from the perspective of the engineering evidence an agent needs. We define a reproducible review protocol with six research questions, five search families and explicit inclusion, quality and coding rules, and we derive an agent-usability framework (AUF) of ten capabilities, from identity to deployability, that links engineering information to agent failure modes. Using AUF, we code 24 standards and vocabularies across five layers of the engineering-semantics landscape and classify industrial agent architectures into five grounding categories. Type and access semantics are widely standardised (11 technologies first class), whereas first-class unit (1), behaviour (4) and constraint (2) semantics are rare, and 0 technologies are first class in all three. A controlled case study illustrates the consequences: exposing these capabilities raises declared-evidence sufficiency from 23% to 86% and fault recall from 68% to 100%; equally informed procedural and SHACL validators reach identical decisions; and matched OPC UA and AAS projections of one plant recover complementary facts, 70% and 57% respectively, while neither carries topology, legal commands, interlocks or observation time. We conclude that agent reliability depends on a coherent runtime evidence contract rather than on a particular syntax, and set out a research agenda for agent-ready industrial standards.**

***Index Terms*—Industrial AI agents, large language models, engineering semantics, semantic interoperability, OPC UA, Asset Administration Shell, knowledge graphs, runtime validation, systematic review.**

I. Introduction

A request such as "raise the fill volume on Line 1 to 330 mL" carries obligations that a fluent answer does not discharge. The agent must identify the filler and its writable setpoint rather than a similarly described measurement, interpret the unit, obtain current evidence, and respect the machine's operating state and process interlocks. These are supervisory decisions above the controller's deterministic loop; an agent's plausible response is not evidence that the proposed command is admissible.

A. Todkar, J. Solanki, and M. Sarkar are with Siemens Foundational Technology, Siemens AG. E-mail: anand.todkar@siemens.com; jitendra.solanki@siemens.com; mrinmoy.sarkar@siemens.com.

Search protocol, screening and coding sheets, and the code that regenerates every number in this paper are released with the article.

Industrial automation has standardised much of the required knowledge over three decades: OPC UA information models [1], the Asset Administration Shell (AAS) [2], property dictionaries [3], [4], ISA-95 hierarchies [5], PackML state machines [6] and the Module Type Package [7]. The semantic web contributes RDF, OWL, SHACL and SPARQL [8]–[11], together with QUDT units [12] and SOSA/SSN observations [13]. These technologies were designed for machine-to-machine interoperability. Whether they supply what a language agent needs at decision time, and in which combination, has not been analysed systematically.

Existing reviews examine industrial language models, industrial knowledge graphs, semantic interoperability and individual standards largely independently, and evaluate technologies against interoperability criteria such as coverage, mapping effort and conformance. Grounding research, conversely, evaluates retrieval and tool use on open-domain data that has no units, interlocks or operational consequences [14]–[17]. The gap is an analysis of the field from the perspective of the *engineering evidence* an agent requires to ground observations and actions.

The central question of this survey is therefore: *what engineering information and semantic capabilities are required to ground trustworthy industrial AI agents, how well do existing industrial standards and semantic technologies provide them, and which gaps remain for deployment?* We contribute:

- **C1.** A reproducible review protocol with six research questions, five search families, explicit eligibility, quality and coding rules, and released screening tooling (Section III).
- **C2.** The agent-usability framework (AUF): ten capabilities, each with a definition, the agent failure it prevents, a scoring rubric and supporting sources, used as the survey's coding instrument (Section IV).
- **C3.** A cross-technology capability analysis of 24 standards and vocabularies organised in a five-layer engineering-semantics landscape (Sections V–VI).
- **C4.** A classification of industrial agent architectures by grounding source and authority, coded with AUF (Section VII), and a reference synthesis derived from it (Section VIII).
- **C5.** A controlled case study showing how capability omissions identified by AUF manifest as grounding and validation failures, including a matched OPC UA/AAS comparison (Section IX), and a research agenda (Section X).

TABLE I
SCOPE OF EXISTING SURVEYS VERSUS THIS SURVEY (✓ CENTRAL, (✓) PARTIAL). AGT.: INDUSTRIAL LLM AGENTS; KG: INDUSTRIAL KNOWLEDGE GRAPHS; IM: INFORMATION MODELS; INT.: SEMANTIC INTEROPERABILITY; VAL.: RUNTIME VALIDATION; FW.: CAPABILITY FRAMEWORK; SYS.: SYSTEMATIC METHOD. ROWS ARE ADDED ONLY AFTER THE AUTHORS VERIFY EACH SURVEY'S FULL TEXT.

Survey	Agt.	KG	IM	Int.	Val.	Fw.	Sys.
This survey	✓	✓	✓	✓	✓	✓	✓

The novelty lies in the analysis perspective, not in any individual technology: we organise the field around the evidence contract that connects engineering information to agent failure modes, and map it simultaneously across standards, semantic technologies and agent architectures.

II. BACKGROUND AND RELATED SURVEYS

Industrial agents. We use *industrial AI agent* for a system in which a language model interprets an operator or system goal, retrieves plant information, and produces answers or typed tool calls that may affect equipment. Tool use and reasoning loops [18], [19] and function-calling benchmarks [17] established the general pattern; protocols such as MCP [20] standardise tool transport, not equipment truth.

Engineering semantics. By *engineering semantics* we mean machine-interpretable statements about plant identity, types, units, structure, behaviour, constraints and provenance, as carried by information models, property dictionaries, ontologies and constraint languages. *Grounding* is the resolution of an utterance or proposal to such statements; *validation* checks a proposal against them before dispatch.

Knowledge-graph grounding. Retrieval-augmented generation conditions output on retrieved evidence [14]; graph-based and hybrid variants retrieve structured neighbourhoods [15], [21], and knowledge-graph/LLM integration has been surveyed [16]. These works mostly extract graphs from text. Industrial evidence such as units, ranges, access rights, state machines and interlocks is rarely written in text and must come from engineering systems. Our earlier IIH Semantics framework [22] integrates OPC UA aggregation, RDF/SPARQL and a generated GraphQL schema on Industrial Edge; this survey generalises the question beyond one architecture.

Related surveys. Reviews of semantic interoperability evaluate standards for machine-to-machine exchange, and reviews of LLMs in manufacturing evaluate applications or models. Table I positions this survey against reviews identified by the protocol of Section III; rows are added only after full-text verification (0 verified at the time of writing). The distinguishing combination is an agent-oriented capability framework applied jointly to standards, semantic technologies and agent architectures with a systematic method.

III. SURVEY METHODOLOGY

A. Research questions

RQ1. Which engineering-information capabilities do industrial AI agents require to ground observations and actions?

RQ2. Which standards, information models, semantic technologies and knowledge representations provide them?

RQ3. Where do current industrial-agent architectures obtain identity, units, topology, behaviour, constraints, provenance and runtime state?

RQ4. Which capabilities remain missing or weakly supported?

RQ5. Which architectural patterns combine LLM reasoning with deterministic engineering validation?

RQ6. What evidence shows that engineering semantics improve grounding, action correctness, transferability, auditability or context efficiency?

RQ1 is answered by the AUF derivation (Section IV), RQ2 by the technology landscape and matrix (Sections V–VI), RQ3–RQ5 by the agent-literature coding (Sections VII–VIII), and RQ6 by the reported evaluations and the case study (Section IX).

B. Sources and search strategy

Four evidence streams are distinguished and never pooled. *Specifications* (IEC, ISO, ISA, VDI/VDE/NAMUR, W3C, ETSI, OPC Foundation, IDTA, ECLASS, GS1, IDSA) are collected from the issuing bodies and scored on what they standardise. *Peer-reviewed literature* published 2021–2026 is searched in IEEE Xplore, Scopus, Web of Science, ACM Digital Library, ScienceDirect and SpringerLink by the authors, and in OpenAlex and Crossref by released tooling; Crossref is relevance-ranked and capped, so it supplements rather than replaces the boolean database searches. *Preprints* (arXiv, posted content) are coded but labelled non-peer-reviewed. *Vendor material* supports only implementation-capability statements. Foundational earlier work (retrieval augmentation, tool use, constraint languages) enters through backward snowballing. Rather than one query, the protocol defines eight topical families, (F1) LLM agents in industrial automation and cyber-physical systems, (F2) industrial knowledge graphs and ontologies with LLMs, (F3) OPC UA, AAS and AutomationML with LLMs or agents, (F4) runtime validation and guardrails in control systems, (F5) semantic grounding and interoperability, (F6) Graph- and KG-RAG on engineering data, (F7) neurosymbolic, ontology-grounded and typed-tool agents, and (F8) safety, provenance and trust, plus a family (R1) dedicated to related surveys. Exact database-specific strings are frozen in protocol v1.1.1, released with the article; forward and backward snowballing from included studies complements them.

C. Eligibility, screening and quality

Studies are included if they address industrial AI or LLM agents or the engineering semantics they consume (I1), industrial information models or knowledge graphs with an AI consumer (I2), grounding, tool use, retrieval or runtime action validation in an engineering context (I3), or constraints, provenance or state models used to check AI outputs (I4). Exclusion codes are: no industrial context (E1); generic RAG or graph construction without engineering semantics (E2); no agent or LLM (E3); duplicate or superseded version (E4); insufficient documentation (E5); inaccessible full text (E6);

language or period (E7). Surveys (I5) feed only the related-survey gap analysis. Records are deduplicated by DOI and by normalised title, year and first author; near-identical titles are listed for manual resolution rather than merged. A deterministic, versioned automation rule then marks records ineligible unless title or abstract contains both an AI/semantics term and an industrial term; every excluded record and the failing rule are released, as PRISMA 2020 requires for automation tools. Title/abstract and full-text screening use spreadsheets that preserve each decision and reason. A seeded random 20% sample is designated for double coding at both stages. Five quality items (architecture described, evaluation method, artefact availability, isolation of the semantic contribution, safety limitations) are scored 0–2 and reported, not used for exclusion.

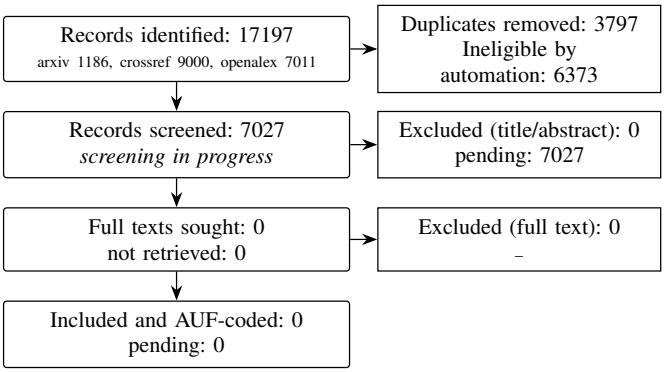


Fig. 1. PRISMA 2020 flow generated from the screening sheets; counts update automatically as screening proceeds.

D. Coding and agreement

Each included system is coded for grounding category (Section VII), retrieval mechanism, tool interface, representation, each AUF capability on a 0–3 scale (absent, implicit, explicit, checked at runtime), runtime validation, controller authority and evaluation level (none, simulation, testbed, physical). Technologies are coded with the rubric in Table II, and each score is linked to a specification reference. Agreement on the double-coded sample is reported as quadratic-weighted Cohen's κ and ordinal Krippendorff's α with adjudicated disagreements; at the time of writing the second rater's coding is pending (κ: pending, α: pending), and all scores are first-rater codes.

E. Capability levels

Every capability statement records its level: *specification* capability (what a standard can express by default), *implementation* capability (what a cited implementation or profile exposes), and *runtime* availability (what the agent actually receives or checks when deciding). Matrix A (Section VI) codes specification capability; Matrix B (Section VII) codes runtime availability and records "not reported" rather than inferring capabilities a paper does not demonstrate.

F. Search status

Figure 1 is generated from the screening sheets. The open-source harvest (2026-10-20) identified 17197 records (9000 Crossref, 1186 arXiv, 7011 OpenAlex; incomplete for: none). After removing 3797 duplicates and marking 6373 records ineligible by the automation rule, 7027 records await title/abstract screening, and 404 industrial survey candidates await full-text verification for Table I. Subscription-database exports are imported with the same tooling, after which all counts are regenerated.

IV. The Agent-Usability Framework

An industrial agent receives a goal, resolves it against a plant it has not seen, decides which evidence it needs, and proposes an answer or action that must be checkable before dispatch. AUF asks what each source of engineering semantics contributes to that cycle. The ten capabilities were derived by tracing the minimum evidence obligations of a bounded supervisory cycle (resolve, interpret, locate, decide, check, trust, retrieve) and the two cross-cutting feasibility conditions of a deployment (context budget and ecosystem). Table II states for each capability the operational requirement, the agent failure it prevents, the scoring anchors and representative sources. The derivation is tested against the coded literature as a saturation check: any recurring requirement that cannot be mapped to U1–U10 is reported as a framework gap.

Three design rules make AUF a repeatable instrument rather than an opinion table. First, capabilities are scored separately, because strength in one does not compensate for another: a `Double` in bar and a `Double` in psi have the same type, and a complete address space need not state the order in which commands are legal. Second, technologies are scored on their default specification, since that is what an agent receives unless a deployment publishes more; deployment profiles are scored separately when agent systems are coded. Third, U1–U8 are evidence obligations whereas U9–U10 are feasibility conditions; AUF does not cover model training, cybersecurity or human factors, and it does not prescribe a serialisation.

V. Engineering-Semantics Technology Landscape

AUF states what an agent needs; the five layers state where capabilities come from. The layers classify sources by role and are not an industrial reference architecture or a strict dependency hierarchy: one standard may contribute to several layers, and a deployment can combine them without materialising one graph. RAMI 4.0 remains the reference architecture; the layers are a complementary evidence view.

L0: Identification and dictionaries. ECLASS, IEC common data dictionaries and IRDIs give properties stable, cross-vendor definitions [3], [4], [23], [27]; GS1 identifiers distinguish products and individual assets. Concept identity is not instance identity: identifying "pressure" does not identify the transmitter on one filler, which needs asset membership and an observation source. Persistent identifiers also need governed mappings when hardware is replaced.

L1: Information and interface models. OPC UA combines typed address spaces, access levels, engineering ranges and callable methods, with companion specifications for domain

TABLE II
AUF CAPABILITIES: REQUIREMENT, FAILURE MODE PREVENTED AND RUBRIC ANCHORS (0 ABSENT; 1 INCIDENTAL; 2 SUPPORTED; 3 FIRST CLASS). SCORES REFLECT WHAT A SPECIFICATION STANDARDISES BY DEFAULT, NOT OPTIONAL EXTENSIONS.

Capability	Requirement	Failure prevented	1 incidental	3 first class	Sources
U1 Identity	resolve phrase to the right concept and instance	wrong equipment; homograph conflation	identifiers unique per vendor or file	globally unique, versioned, resolvable, synonym handling	[3], [4], [23]
U2 Types and access	know datatypes, ranges, access rights	malformed call; write to read-only point	datatype only	datatypes, ranges, access rights, enumerations, run-time introspection	[1], [17]
U3 Quantities	bind values to quantity kind and unit	dimension or scale error	unit as display string	quantity kind, dimension vector, affine conversion	[12]
U4 Relations	containment, topology, typed relations	wrong scope; missed impact	containment only	typed relations with transitivity and inverses	[5], [24]
U5 Behaviour	operations, state machines, legal commands	command illegal in current state	operations named, untyped	typed operations plus state model with legality conditions	[6], [7]
U6 Constraints	checkable guards and envelopes	unsafe or out-of-envelope proposal	range or type checks	executable constraint language with reports	[10], [25]
U7 Provenance and time	timestamped, attributed evidence	stale or untraceable evidence	one global timestamp	per-value time, source, agent and derivation	[13], [26]
U8 Query pushdown	filter, join, aggregate outside the model	context stuffing; unanswerable aggregates	filtered fetch	query language with joins, aggregation, path traversal	[11], [15]
U9 Context economy	faithful minimal evidence slice	budget exhaustion before evidence	verbose, partial fetch	compact and projectable to a minimal faithful slice	[14], [21]
U10 Ecosystem	implementations, converters, governance	deployment never happens	one implementation	open implementations, converters, reference data, active governance	[2], [20]

content [1]; the AAS supplies submodels with semantic identifiers [2], [28]. AutomationML carries engineering exchange [29], ISA-95 equipment hierarchies [5], and MTP and PackML services and state machines [6], [7]; WoT Thing Descriptions, NGSI-LD and IEC 61850 provide further interface and entity models [30]–[32]. These models make introspection and tool synthesis possible, but what a deployment exposes must be audited: an agent cannot check an interlock that was never published, even when the host standard could express it.

L2: Formal semantics, query and validation. RDF provides globally named relations, OWL entailment over classes and properties, SHACL closed-world checks of a data graph against shapes, and SPARQL joins, traversal and aggregation outside the model [8]–[11]; PROV-O records derivation and responsibility [26]. Open-world consistency does not enforce mandatory action parameters, and shape conformance does not establish that the engineering mapping or the set of interlocks is complete.

L3: Quantities, observations and domain relations. QUDT supplies quantity kinds, dimension vectors and conversion factors [12]; a display symbol such as "bar" does not itself state a conversion. SOSA/SSN separates sensors, observed properties, observations and results [13], so that an answer can cite the measurement and time it reports. Brick, SAREF and DEXPI contribute typed equipment and process relations [24], [33], [34]; flow topology supports impact analysis but not causal diagnosis without incident evidence.

L4: Cross-organisational policy and trust. Dataspaces add participant identity, catalogues, contracts and usage policies [35], [36]; ODRL expresses permissions and duties [25]. Policy is a gate separate from physical operating constraints, and an ODRL statement does not by itself enforce downstream use.

VI. COMPARATIVE CAPABILITY ANALYSIS

Table III codes 24 technologies (240 cells). Each row is linked to its specification; clause-level locators are being added to the released evidence register (0 of 240 cells located, 1 row awaiting a specification reference). Reading the matrix by column yields four observations.

Typing is solved; consequential semantics are sparse. 11 technologies are first class on U2, which explains why "AI-ready" data layers that expose types appear convincing. First-class support is rare for units (1), behaviour (4) and constraints (2), the capabilities that separate a well-typed proposal from an admissible one.

No single technology provides the evidence contract. 0 technologies are first class on all of U3, U5 and U6, and only 1 (MTP) reaches at least "supported" on all three. An agent that needs unit, behaviour and constraint evidence must therefore compose standards, for example an information model for types and states, a unit ontology for conversion and a constraint language for guards.

Richness and economy trade off. The richest information models score lowest on context economy (U9). This is not a defect of the standards but indicates that semantic models belong behind query and projection interfaces rather than in prompts (U8).

Provenance is available but separate. First-class provenance (5 technologies) concentrates in observation, provenance, context-entity and dataspace models, whereas the machine-interface models OPC UA and AAS are only supported on U7. Freshness checks therefore require joining layers at runtime.

These observations concern what specifications standardise. Deployed profiles can expose more (OPC UA methods with state machines) or less (an AAS without constraint submodels), which is why agent systems are coded on what they actually use (Section VII) and why the case study compares emitted profiles (Section IX).

TABLE III
AUF CODING OF 24 STANDARDS AND VOCABULARIES (SCORES 0–3, RUBRIC IN TABLE II). SCORES DESCRIBE WHAT THE SPECIFICATION STANDARDISES BY DEFAULT, NOT DEPLOYED PROFILES; SINGLE-RATER (R1) CODING, INTER-RATER AGREEMENT PENDING. †SPECIFICATION REFERENCE TO BE ADDED.

L	Technology	U1	U2	U3	U4	U5	U6	U7	U8	U9	U10
L0	ECLASS / IEC CDD / IEC 61987 [3], [4], [23], [27]	3	2	2	1	0	1	0	1	2	3
	GS1 identifiers / EPCIS †	3	1	0	1	1	0	2	1	2	3
L1	OPC UA core and companions [1]	3	3	2	2	3	1	2	1	1	3
	Asset Administration Shell [2], [28]	3	3	2	2	2	1	2	1	1	3
	AutomationML [29]	2	3	1	3	1	1	1	0	1	2
	MTP [7]	2	3	2	2	3	2	1	0	2	2
	PackML [6]	1	2	0	0	3	2	0	0	3	3
	ISA-95 / IEC 62264 [5]	2	2	1	3	1	0	1	0	2	3
	W3C WoT TD 1.1 [30]	3	3	2	1	3	1	1	1	2	2
	NGSI-LD [31]	3	2	2	3	1	1	3	2	2	2
	IEC 61850 [32]	3	3	2	2	2	1	2	0	1	3
L2	RDF / RDFS [8]	3	2	0	3	0	0	1	3	1	3
	OWL 2 [9]	3	3	0	3	0	2	0	2	1	3
	SHACL [10]	2	3	1	2	1	3	1	2	2	3
	SPARQL 1.1 [11]	2	1	0	3	0	1	1	3	3	3
	PROV-O [26]	3	2	0	3	1	0	3	2	2	2
L3	QUDT [12]	3	2	3	1	0	2	0	2	2	2
	SOSA / SSN [13]	3	2	1	2	1	0	3	2	2	3
	Brick [24]	3	3	1	3	0	1	0	3	2	3
	SAREF and extensions [33]	3	2	2	2	2	0	1	2	2	2
	DEXPI [34]	2	3	2	3	0	1	1	1	1	2
L4	IDS-RAM / Dataspace Protocol [35]	3	2	0	2	1	2	3	1	1	2
	Catena-X / Manufacturing-X [36]	3	3	1	2	1	2	3	1	1	2
	ODRL [25]	2	2	0	2	2	3	2	1	2	2

	Pipeline
A	LLM → documents, manuals → recommendation
B	LLM → typed tools → industrial API
C	LLM → knowledge-graph retrieval → engineering context
D	LLM → proposal → engineering validation → execution
E	LLM → validation → dispatch gate → controller checks → process

Fig. 2. Grounding categories used to code industrial agent systems. Categories are cumulative in authority, not in capability: a C system may lack typed tools.

VII. STATE OF THE ART IN INDUSTRIAL AGENT GROUNDING

Published industrial agents differ less in the language model they use than in where their engineering evidence comes from and who holds authority over actions. We therefore classify systems into five grounding categories (Fig. 2) and code each included system with AUF on what it actually uses at runtime (Table IV).

A: Document-grounded. Retrieval over manuals, procedures and logs [14] grounds explanations and troubleshooting. Identity (U1) and units (U3) remain implicit in text, so answers can be fluent yet unverifiable, and actions have no checkable target.

B: Typed-tool grounded. The model calls declared functions on OPC UA, AAS or vendor APIs [17]–[20]. Schemas provide types and access (U2) and can constrain identifiers, but units, state legality and guards are only as explicit as the exposed interface; tool transport does not establish equipment truth.

C: Knowledge-graph grounded. Retrieval over engineering graphs supplies identity, relations and query pushdown (U1, U4, U8) [15], [16], [21], [22]. Graphs built from engineering systems carry facts that text-derived graphs cannot, but retrieval quality and the completeness of the mapped constraints become new dependencies.

D: Semantically validated. Proposals are lifted into a checkable form and validated against constraints and state models (U5, U6) before execution [10]. Validation converts a claim into a proposition with a truth value, but only relative to the constraints that were modelled.

E: Controller-authoritative. Validation is repeated at dispatch with fresh evidence (U7), and the controller retains independent, current-state checks and final actuator authority. This category separates supervisory reasoning from deterministic control and is the only one compatible with functional-safety practice; it is also the least reported with physical evaluation.

The coding records, per system, which AUF capabilities are explicit or runtime-checked, whether validation and controller authority exist, and the evaluation level. The analysis questions are fixed in advance: which capabilities are most often absent (RQ4), which category combinations co-occur with runtime validation (RQ5), and which studies report measured effects of engineering semantics on grounding, action correctness or context cost rather than demonstrations (RQ6). The number of coded systems (0 at the time of writing) and their distribution over categories are regenerated from the released coding sheet.

VIII. A REFERENCE SYNTHESIS

Fig. 3 synthesises the surveyed patterns into a reference flow. It is derived from the capability analysis and the grounding categories, not a claim that every industrial agent must use RDF

TABLE IV
AUF CODING OF INDUSTRIAL AI-AGENT SYSTEMS (0 ABSENT, 1 PROSE/IMPLICIT, 2 EXPLICIT, 3 CHECKED AT RUNTIME). CAT.: GROUNDING CATEGORY A–E (FIG. 2).

System	Cat.	U1	U2	U3	U4	U5	U6	U7	U8	U9	U10	Valid.	Eval.
Full-text coding of included systems pending (Sec. III).													

or SHACL: any representation that delivers the same evidence and checks satisfies it, including competent procedural code.

Four properties follow from the analysis. (i) Validation distinguishes three dispositions: reject when a violation is proved, abstain when required evidence is missing, and accept only relative to the checked evidence contract. Absent evidence is not evidence of an illegal command. (ii) A proposal accepted at planning time is revalidated at dispatch, because state and observations change. (iii) Repair is bounded, names the violated rule, and never weakens a guard or widens authorisation to avoid refusal. (iv) The semantic model stays behind authenticated, parameterised query interfaces with versioned mappings rather than being placed in the prompt or exposed as unrestricted graph or PLC access.

IX. CONTROLLED CASE STUDY

The case study asks whether capability gaps identified by AUF manifest as observable grounding and validation failures. It uses an authored two-line bottling plant (17 assets, 73 signals, 22 shared descriptions, metric and imperial units, four interlocks, 14 feed relations) with 35 tasks that declare their required facts and 137 proposed actions (84 injected faults in 12 families, 53 legal). Context conditions C0–C4 expose progressively more capabilities: tags and values; descriptions and unit symbols; a typed OPC UA/AAS-like profile; units, topology and taxonomy; and behaviour, guards and provenance. Validator tiers G0–G4 check the same capability sets. Sufficiency is the fraction of tasks whose declared facts are all present within a 6,000-token budget (deterministic surrogate tokenizer); it is not a bound on model accuracy. Tasks, faults and validators share one authored specification, so the study illustrates mechanisms and is not an external validation.

CS1: Progressive exposure. Top-1 grounding rises from 79% with descriptions to 95% with a typed profile, yet sufficiency at C2 is only 23%, rising to 86% at C4 (Table V). Validator recall rises from 68% at G2, which also raises false alarms on 8% of legal actions through literal unit comparison, to 100% at G4 without false alarms. Typing resolves the target but not whether the action is admissible.

CS2: Retrieval. At equal budget, graph traversal yields the highest sufficiency where relations exist, membership-scoped retrieval is cheapest but omits evidence, and lexical retrieval cannot compensate for capabilities the context lacks (Table VI). Evidence availability and retrieval are separate variables.

CS3: Capability removal. Removing one capability at a time reduces sufficiency, recall or both, and turns detections into abstentions on missing evidence (Table VII). Removing unit semantics leaves task sufficiency unchanged in this budget yet causes the largest loss of fault recall: the effect of a capability depends on whether it is needed to answer or to check.

CS4: Representation versus information. A procedural validator and a SHACL validator given identical engineering facts produce 0 raw-family disagreements (Table VIII). The measured benefit lies in explicit engineering evidence, not in the validation technology; reuse, auditability and maintenance effort remain to be measured.

CS5: OPC UA versus AAS. The plant's OPC UA NodeSet2 and AAS v3 projections were parsed back into 726 matched facts (Table IX). The OPC UA profile recovers 507 (70%), including access levels, engineering ranges and hierarchy; the AAS profile recovers 411 (57%), including semantic identifiers, nameplates, operating state and values; their union reaches 626 (86%). OPC UA `EUInformation` is attached to the unit, so 20 count-type signals report the unit's quantity kind rather than the signal's. Neither projection carries flow topology, legal commands, interlocks or observation timestamps, which C4 requires. The serialisations cost 52,324 and 70,871 surrogate tokens. Both standards can express more through companion models, qualifiers or additional submodels; the result shows that the emitted profiles are complementary and jointly incomplete, not that either standard is inferior. A second, derived pilot, an IIH-attributed KUKA robot NodeSet and its loss-aware AAS projection, retains all 676 of 676 source facts, including 90 nodes and 277 references; it verifies conversion fidelity only.

X. OPEN CHALLENGES AND RESEARCH AGENDA

Semantic completeness. Validation is only as complete as the modelled guards. Methods are needed to show that an interlock inventory matches the control logic, for example by extracting guards from PLC programs and cross-checking them with declared constraints.

Semantic correctness. A graph can conform to every shape and still be wrong. Unit consistency with process equations, declared topology versus P&IDs, and the equivalence of constraint models and ladder logic are decidable checks that are not yet routine. An incorrect semantic layer is more dangerous than none because it is trusted.

Runtime freshness and dispatch races. Engineering semantics combined with stale observations remain unsafe. Agents need freshness contracts, revalidation at dispatch and measured behaviour under reconnects, subscription lag and state changes between planning and execution.

Agent and controller authority. The boundary between supervisory reasoning and deterministic control needs explicit design: which checks the gateway performs, which remain in the controller, and how rejections and acknowledgements are traced.

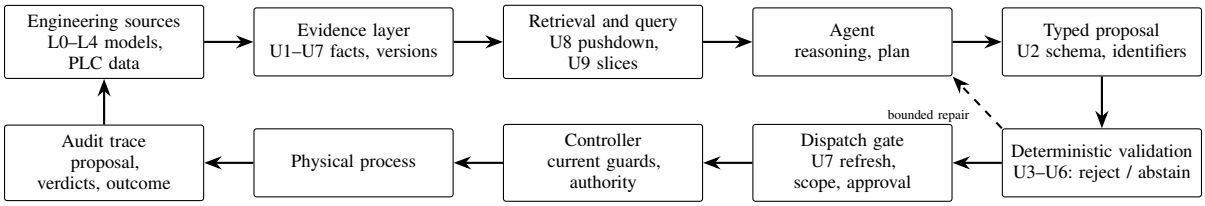


Fig. 3. Reference synthesis of runtime grounding. Evidence and validation support the agent; dispatch and controller checks retain current-condition authority. No language model participates in a hard-real-time loop.

TABLE V
CS1: PROGRESSIVE CAPABILITY EXPOSURE ON THE AUTHORED REFERENCE PLANT (17 ASSETS, 73 SIGNALS, 35 TASKS, 84 FAULTY AND 53 LEGAL PROPOSALS). SUFF.: DECLARED REQUIRED FACTS PRESENT AT 6,000 SURROGATE TOKENS (NOT AN ACCURACY BOUND). CONTEXT Ck IS PAIRED WITH VALIDATOR Gk.

Ctx.	Projection	Top-1	Verif.	Suff.	Val.	Recall	FPR
C0	OPC DA / PLC symbol table	0%	0%	0%	G0	0%	0%
C1	SCADA/historian CSV	79%	0%	0%	G1	14%	0%
C2	OPC UA NodeSet2 / AAS submodels	95%	95%	23%	G2	68%	8%
C3	RDF/OWL + QUDT + Brick + ISA-95	100%	100%	51%	G3	74%	0%
C4	PackML + interlocks + SOSA/PROV	100%	84%	86%	G4	100%	0%

TABLE VI
CS2: RETRIEVAL STRATEGY CROSSED WITH EXPOSED CAPABILITIES AT 6,000 SURROGATE TOKENS. MEMBERSHIP AND GRAPH RETRIEVAL SHARE A FIXED ENTITY RESOLVER; GRAPH TRAVERSAL REQUIRES C3/C4 RELATIONS.

Ctx.	Retrieval	Suff.	Coverage	Tokens
C2	lexical	23%	55%	5193
C2	membership	23%	48%	1906
C3	lexical	40%	67%	5075
C3	membership	29%	59%	2507
C3	graph	46%	76%	5341
C4	lexical	71%	85%	5107
C4	membership	46%	69%	2767
C4	graph	80%	93%	5345

TABLE VII
CS3: REMOVING ONE AUF CAPABILITY AT A TIME FROM THE FULL C4/G4 CONFIGURATION. ABSTENTIONS ON MISSING EVIDENCE ARE NOT COUNTED AS DETECTIONS.

Removed	Suff.	Recall	Abstain
none	80%	100%	0
behaviour	63%	89%	37
constraints	69%	96%	20
provenance	74%	85%	40
topology	63%	94%	5
units	80%	61%	60

TABLE VIII
CS4: EQUALLY INFORMED PROCEDURAL AND SHACL VALIDATORS ON IDENTICAL ENGINEERING FACTS.

Validator	Faults caught	False alarms	Diagnosis
Procedural	84/84	0/53	100%
SHACL	84/84	0/53	100%

TABLE IX
CS5: MATCHED FACTS RECOVERED FROM THE EMITTED OPC UA NODESET2 AND AAS V3 PROJECTIONS OF THE SAME PLANT ($\neq$: CONFLICTING VALUE). BOTH PROJECTIONS ARE GENERATED FROM ONE SPECIFICATION, SO THIS MEASURES THE EMITTED PROFILES, NOT LIMITS OF EITHER STANDARD.

Fact	AUF	OPC UA	AAS	Union
identity	U1	73/73	73/73	73/73
semantic id	U1	0/46	46/46	46/46
nameplate	U1	0/11	11/11	11/11
datatype	U2	73/73	73/73	73/73
access	U2	73/73	0/73	73/73
range	U2	73/73	0/73	73/73
unit	U3	73/73	73/73	73/73
quantity kind	U3	53/73 (20$\neq$)	0/73	53/73 (20$\neq$)
membership	U4	73/73	73/73	73/73
parent	U4	16/16	0/16	16/16
feeds	U4	0/14	0/14	0/14
state	U5	0/9	9/9	9/9
legal commands	U5	0/9	0/9	0/9
interlock	U6	0/4	0/4	0/4
value	U7	0/53	53/53	53/53
observation time	U7	0/53	0/53	0/53

Model evolution and transfer. PLC programs, configurations and mappings change. How semantic constraints are versioned, migrated and transferred between machines, and at what engineering cost, is largely unmeasured; reuse and maintainability claims for declarative constraints require change-effort studies.

Engineering cost and ownership. Someone must author and maintain identity maps, unit bindings, state models and guards. Bootstrapping from existing artefacts (OPC UA address spaces, AutomationML, DEXPI, PLC projects) with formal acceptance checks is a promising route.

Benchmarks with actions and consequences. Industrial grounding lacks shared benchmarks with real installations, independently authored tasks, labelled faulty and legal actions, severity-weighted scoring and physical outcomes; the synthetic corpus used here is a starting point, not a substitute.

Safety assurance. Semantic validation is not functional safety. Zero observed invalid actions in a test corpus establishes no safety integrity level; assurance cases must separate proposal quality, gateway rejection and controller protection.

Authorisation and policy. Agents need the analogue of operator competence and shift authorisation, joining policy languages and security frameworks [25], [37] to equipment models and plant states.

Agent-ready standards. Existing standards were designed for interoperability. Publishing constraints and legal-command models alongside information models, for instance as a versioned constraint submodel or companion specification, would close the largest gap identified by the capability analysis.

XI. Threats to Validity and Conclusion

Threats to validity. The search may miss grey literature and vendor work, and arXiv records are not peer reviewed; snowballing and labelling mitigate but do not remove this. AUF scores are first-rater codes based on default specifications until the second rater's agreement is reported, and optional extensions or deployed profiles can change judgements. The capability list was derived top-down and is tested for saturation against the coded corpus. The case study uses one authored plant whose tasks, faults and validators share a specification; its OPC UA/AAS comparison measures emitted profiles from that specification. No physical trial or complete model evaluation is reported; the surrogate tokenizer does not measure billing tokens.

Conclusion. Industrial AI-agent reliability depends less on the choice of language model or semantic syntax than on whether the agent has access to explicit, current, machine-interpretable engineering evidence covering identity, quantities, relations, behaviour, constraints and provenance. Existing standards provide many of these capabilities, but they are fragmented across technologies and rarely exposed as a coherent runtime evidence contract. The case study shows the consequences: typing alone resolves targets but not admissibility, equally informed validators decide identically, and matched OPC UA and AAS profiles are complementary yet jointly incomplete. Making industrial standards agent-ready, chiefly by publishing behaviour and constraints with provenance behind query interfaces, is the most consequential next step.

References

[1] International Electrotechnical Commission, "IEC 62541: OPC unified architecture," IEC, Tech. Rep., 2020, multi-part standard; information model in Part 3, address space model in Part 5.

[2] Industrial Digital Twin Association, "Specification of the asset administration shell — part 1: Metamodel; part 2: Application programming interfaces; part 3a: Data specification IEC 61360; part 5: Package file format (AASX)," IDTA, Tech. Rep., 2023. [Online]. Available: https://industrialdigitaltwin.org/

[3] ECLASS e.V., "ECLASS — the cross-industry product data standard," ECLASS e.V., Tech. Rep., 2024. [Online]. Available: https://eclass.eu/

[4] International Electrotechnical Commission, "IEC 61360: Standard data element types with associated classification scheme," IEC, Tech. Rep., 2017, basis of the IEC Common Data Dictionary (CDD).

[5] ——, "IEC 62264: Enterprise-control system integration (ISA-95)," IEC, Tech. Rep., 2013.

[6] International Society of Automation, "ANSI/ISA-88 batch control and ANSI/ISA-TR88.00.02 (PackML) machine and unit states," ISA, Tech. Rep., 2008.

[7] VDI/VDE/NAMUR, "VDI/VDE/NAMUR 2658: Automation engineering of modular systems in the process industry — module type package," VDI, Tech. Rep., 2019.

[8] R. Cyganiak, D. Wood, and M. Lanthaler, "RDF 1.1 concepts and abstract syntax," W3C, W3C Recommendation, 2014. [Online]. Available: https://www.w3.org/TR/rdf11-concepts/

[9] W3C OWL Working Group, "OWL 2 web ontology language document overview (second edition)," W3C, W3C Recommendation, 2012. [Online]. Available: https://www.w3.org/TR/owl2-overview/

[10] H. Knublauch and D. Kontokostas, "Shapes constraint language (SHACL)," W3C, W3C Recommendation, 2017. [Online]. Available: https://www.w3.org/TR/shacl/

[11] S. Harris and A. Seaborne, "SPARQL 1.1 query language," W3C, W3C Recommendation, 2013. [Online]. Available: https://www.w3.org/TR/sparql11-query/

[12] QUDT.org, "QUDT — quantities, units, dimensions and data types ontologies," https://qudt.org/, 2024.

[13] A. Haller, K. Janowicz, S. J. D. Cox, M. Lefrançois, K. Taylor, D. Le Phuoc, J. Lieberman, R. García-Castro, R. Atkinson, and C. Stadler, "The modular SSN ontology: A joint W3C and OGC standard specifying the semantics of sensors, observations, sampling and actuation," *Semantic Web*, vol. 10, no. 1, pp. 9–32, 2019.

[14] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[15] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, and J. Larson, "From local to global: A graph RAG approach to query-focused summarization," arXiv:2404.16130, 2024. [Online]. Available: https://arxiv.org/abs/2404.16130

[16] S. Pan, L. Luo, Y. Wang, C. Chen, J. Wang, and X. Wu, "Unifying large language models and knowledge graphs: A roadmap," *IEEE Transactions on Knowledge and Data Engineering*, 2024.

[17] S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez, "Gorilla: Large language model connected with massive APIs," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[18] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[19] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations (ICLR)*, 2023.

[20] Anthropic, "Model context protocol specification," https://modelcontextprotocol.io/, 2024.

[21] B. Sarmah, B. Hall, R. Rao, S. Patel, S. Pasquali, and D. Mehta, "HybridRAG: Integrating knowledge graphs and vector retrieval augmented generation for efficient information extraction," arXiv:2408.04948, 2024. [Online]. Available: https://arxiv.org/abs/2408.04948

[22] A. Todkar, M. Sarkar, and J. Solanki, "Semantic framework for IT-OT integration in industrial environments," *Annual Conference of the PHM Society*, vol. 17, no. 1, 2025. [Online]. Available: https://www.papers.phmsociety.org/index.php/phmconf/article/view/4551

[23] International Organization for Standardization, "ISO 29002-5: Industrial automation systems and integration — exchange of characteristic data — part 5: Identification scheme," ISO, Tech. Rep., 2010.

[24] B. Balaji, A. Bhattacharya, G. Fierro, J. Gao, J. Gluck, D. Hong, A. Johansen, J. Koh, J. Ploennigs, Y. Agarwal, M. Berges, D. Culler, R. Gupta, M. B. Kjærgaard, M. Srivastava, and K. Whitehouse, "Brick: Towards a unified metadata schema for buildings," in *Proceedings of the 3rd ACM International Conference on Systems for Energy-Efficient Built Environments (BuildSys)*, 2016.

[25] R. Iannella and S. Villata, "ODRL information model 2.2," W3C, W3C Recommendation, 2018. [Online]. Available: https://www.w3.org/TR/odrl-model/

[26] T. Lebo, S. Sahoo, and D. McGuinness, "PROV-O: The PROV ontology," W3C, W3C Recommendation, 2013. [Online]. Available: https://www.w3.org/TR/prov-o/

[27] International Electrotechnical Commission, "IEC 61987: Industrial-process measurement and control — data structures and elements in process equipment catalogues," IEC, Tech. Rep., 2019.

[28] Industrial Digital Twin Association, "Submodel templates of the asset administration shell: Digital nameplate and technical data," IDTA, Tech. Rep., 2023.

[29] International Electrotechnical Commission, "IEC 62714: Engineering data exchange format for use in industrial automation systems engineering — AutomationML," IEC, Tech. Rep., 2018.

[30] W3C Web of Things Working Group, "Web of things (WoT) thing description 1.1," W3C, W3C Recommendation, 2023. [Online]. Available: https://www.w3.org/TR/wot-thing-description11/

[31] ETSI, “ETSI GS CIM 009: Context information management (CIM); NGSI-LD API,” ETSI, Tech. Rep., 2023.
[32] International Electrotechnical Commission, “IEC 61850: Communication networks and systems for power utility automation,” IEC, Tech. Rep., 2020.
[33] ETSI, “ETSI TS 103 264: SmartM2M; smart applications; reference ontology and oneM2M mapping (SAREF),” ETSI, Tech. Rep., 2020.
[34] DEXPI Initiative, “DEXPI p&ID specification,” https://dexpi.org/, 2023.
[35] International Data Spaces Association, “IDS reference architecture model, version 4,” IDSA, Tech. Rep., 2022. [Online]. Available: https://internationaldataspaces.org/
[36] Catena-X Automotive Network, “Semantic aspect meta model (SAMM) and catena-x standards,” https://catena-x.net/, 2024.
[37] International Electrotechnical Commission, “IEC 62443: Security for industrial automation and control systems,” IEC, Tech. Rep., 2021.